

```
# loss plotting: Taining vs Validation
loss_values = his_dict['loss']
val_loss_values = his_dict['val_loss']
epochs = range(1, len(his_dict['acc'])
+ 1)
plt.plot(epochs, loss_values, 'b',
label='Training Loss')
plt.plot(epochs, val_loss_values, 'g',
label='Validation Loss')
plt.title('Training & Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

```
plt.clf() ## clear the plot
```

```
# Accuracy plotting: Training vs
Validation
acc_values = his_dict['acc']
val_acc_values = his_dict['val_acc']
plt.plot(epochs, acc_values, 'b',
label='Training Accuracy')
plt.plot(epochs, val_acc_values, 'g',
label='Validation Accuracy')
plt.title('Training & Validation
Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

The output is shown in Figure 6. For testing over test data (10k samples), use the following code:

```
# Now it's QA time! Test it over 10k
samples.
```

```
test_loss, test_acc = network.
evaluate(test_images, test_labels)
print('The accuracy I am expecting:',
test_acc);
```

The output is:

```
10000/10000
[=====
] - 1s 61us/step
The accuracy I am expecting: 0.9752
```

The code for prediction is given

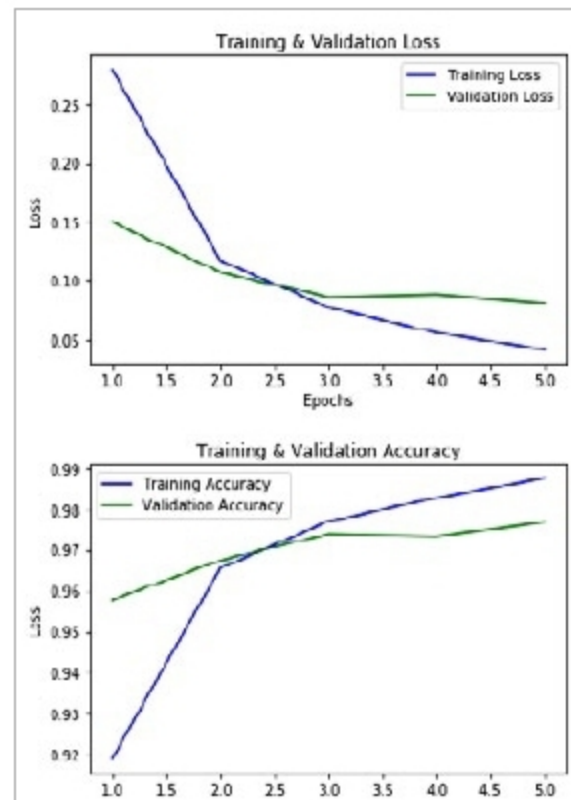


Figure 6: MNIST data set training vs validation plotting

below:

```
# Prediction section
```

```
predictions = network.predict(test_
images);
print (predictions.shape);
print ('Probablistic distribution
of the first digit prediction: \n' +
str(predictions[0]));
```

```
# See what we got?
predicted_value =
np.argmax(predictions[0]);
print ('Predicted pic: ' +
str(predicted_value));
print ('Actual picture under test: ' +
str(np.argmax(test_labels[0])));

print ('Bingo!')
```

The output is:

```
(10000, 10)
Probablistic distribution of the first
digit prediction:
[2.10214646e-09 1.07541705e-10
1.82529277e-06 8.29521523e-05
7.31067290e-12 4.73346908e-08
1.90508078e-13 9.99913931e-01
2.09084714e-08 1.17789079e-06]
Predicted pic: 7
Actual picture under test: 7
Bingo!
```

Universal approach

The steps needed to attack and solve any DL problem are listed in Table 2.

Using Google Colab

Many of our laptops do not have GPUs,

Define the problem	What data is available and what am I going to predict (data science pass)?
Evaluation	How am I going to measure success?
Preparedness	I have 70k data points. What will be my training data, validation data and test data? Will it be equally divided, or a K folding, or iterative K folding?
Tensoring	Process the data such that one can represent it as tensors. The network can only understand a tensor.
Model build up	Build up the first model that beats trivial common sense.
Model refinement	Run it though iterations to fine tune the weights, in order to minimise the loss function and increase the measured success.
Avoid overfitting	Make sure the model is not 'over specialised' for validation data, such that it starts behaving in an ad hoc way when unknown data is fed into the network.
Periodic retraining	As and when new data points are available, keep on re-training the network.

Table 2: Universal deep learning workflow